DECODE LABS

IoT DEPARTMENT | INTERNSHIP PROGRAM

WEEK 3 PROJECT

Project 3: Cloud-Connected Security Node

(IoT Telemetry over Wi-Fi & MQTT)

Wi-Fi Stack • MQTT Protocol • Cloud Dashboard

Name: Noor Fatima

Intern: IoT Department

Company: Decode Labs

Date: July 04, 2026

Project 3: Cloud-Connected Security Node (IoT Telemetry)

1. Project Goal

Broadcast physical security telemetry across the internet to a centralized cloud dashboard. The system uses an ultrasonic distance sensor to monitor a protected zone, connects the microcontroller to a local Wi-Fi network, and streams live distance readings to a free IoT cloud dashboard using a lightweight publish/subscribe protocol (MQTT), allowing the security status to be monitored remotely in real time.

2. Key Requirements

- Connect the microcontroller to a local Wi-Fi network stream.
- Use a lightweight protocol (MQTT or HTTP POST) to publish sensor data (e.g., an ultrasonic distance sensor tracking an intruder).
- Display the live data stream visually on a free IoT cloud dashboard (like Adafruit IO or Blynk).
- Continuously repeat the sense → publish → display cycle for real-time monitoring.

3. Key Skills Demonstrated

- **Wi-Fi Stack Configuration** — initializing the microcontroller's onboard radio, joining a local network, and managing connection state.
- **MQTT / HTTP Telemetry Protocols** — publishing structured sensor data to a remote broker using a lightweight, low-bandwidth publish/subscribe protocol.
- **Cloud Dashboard Integration** — binding device data feeds to a live, remotely-accessible visual dashboard (Adafruit IO).

4. Components Used

Component	Role in Circuit	Connected To
ESP32 Dev Board	Wi-Fi + MQTT enabled microcontroller / Brain of the system	All modules
HC-SR04 Ultrasonic Sensor	Measures distance to detect an approaching intruder	GPIO17 (TRIG) / GPIO16 (ECHO)
Status LED + 220Ω Resistor	Visual local alert indicator when threshold is breached	GPIO18
Breadboard	Distributes shared 3V3 and GND rails	ESP32 + Sensor + LED
Wi-Fi Router	Provides local network / internet access	ESP32 (wireless)
Adafruit IO (Cloud MQTT Broker)	Receives published telemetry and renders the live dashboard	Internet (MQTT over Wi-Fi)

5. Circuit Diagram (Tinkercad-Style Simulation)

The circuit below was laid out in a Tinkercad-style breadboard view. The ESP32 Dev Board supplies 3V3/GND to the breadboard's power rails, which power the HC-SR04 ultrasonic sensor. GPIO17 drives the sensor's TRIG pin to emit an ultrasonic pulse, and GPIO16 reads the ECHO pin to time the pulse's return. GPIO18 drives a local status LED (through a 220Ω resistor) as an on-device alert indicator, while the ESP32's built-in radio maintains the Wi-Fi link used to publish telemetry to the cloud.

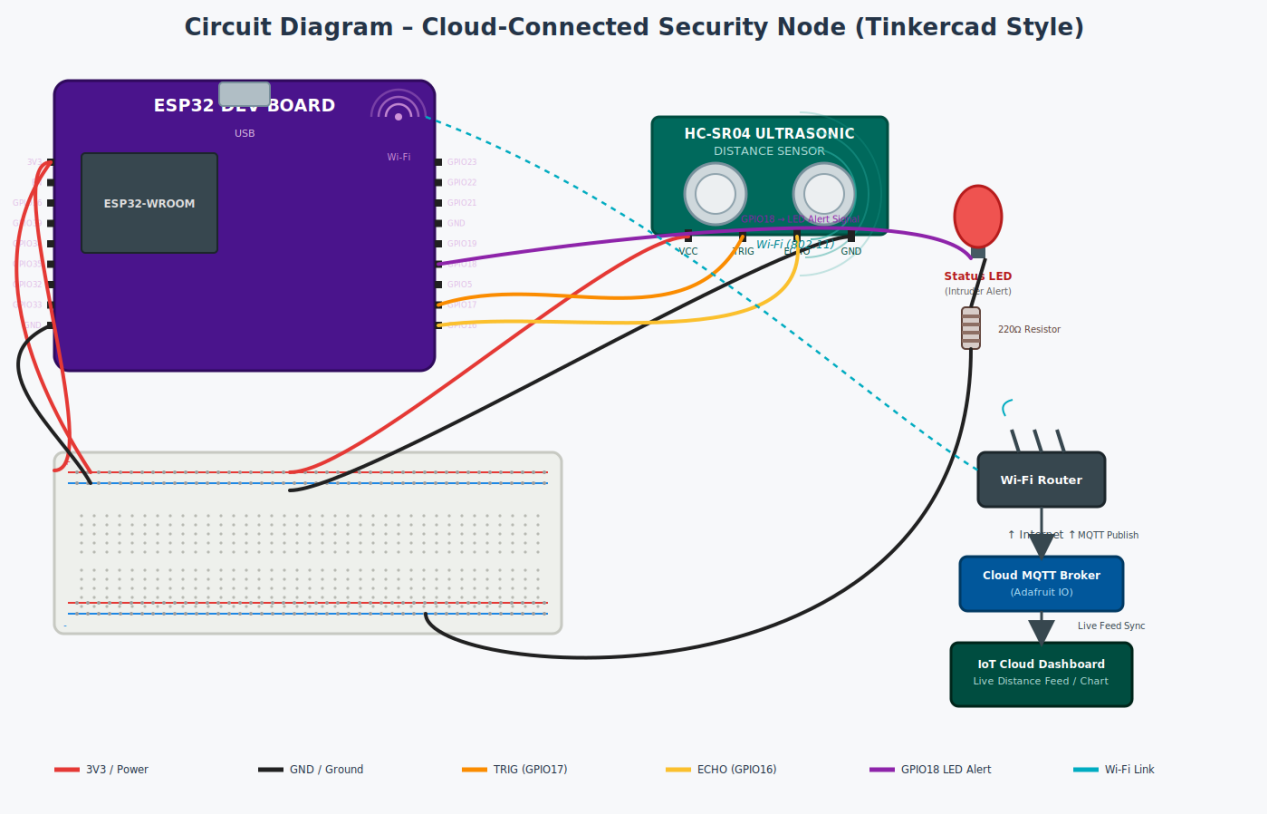

Figure 1: Full circuit wiring — ESP32 Dev Board, HC-SR04 Ultrasonic Sensor, Status LED, and Wi-Fi/MQTT link to the cloud dashboard.

5.1 Wiring Summary

Signal	From	To	Wire Color
Power (3V3)	ESP32 3V3 pin	Breadboard (+) rail	Red
Ground	ESP32 GND pin	Breadboard (–) rail	Black
Sensor Power	Breadboard (+) rail	HC-SR04 VCC	Red
Sensor Ground	Breadboard (–) rail	HC-SR04 GND	Black
Trigger Signal	ESP32 GPIO17	HC-SR04 TRIG	Orange
Echo Signal	ESP32 GPIO16	HC-SR04 ECHO	Yellow
Alert Signal	ESP32 GPIO18	LED Anode (via 220Ω resistor)	Purple

Wireless Link	ESP32 Wi-Fi radio	Local Router → Internet → Adafruit IO	Wi-Fi (802.11)

6. Telemetry Flow: Device to Cloud Dashboard

The node executes a continuous sense-publish cycle. After joining the local Wi-Fi network and connecting to the Adafruit IO MQTT broker, the ESP32 repeatedly triggers the ultrasonic sensor, calculates distance from the echo pulse width, evaluates it against an intruder threshold, and publishes the result (plus an alert flag when triggered) to the cloud, where Adafruit IO renders it as a live chart.

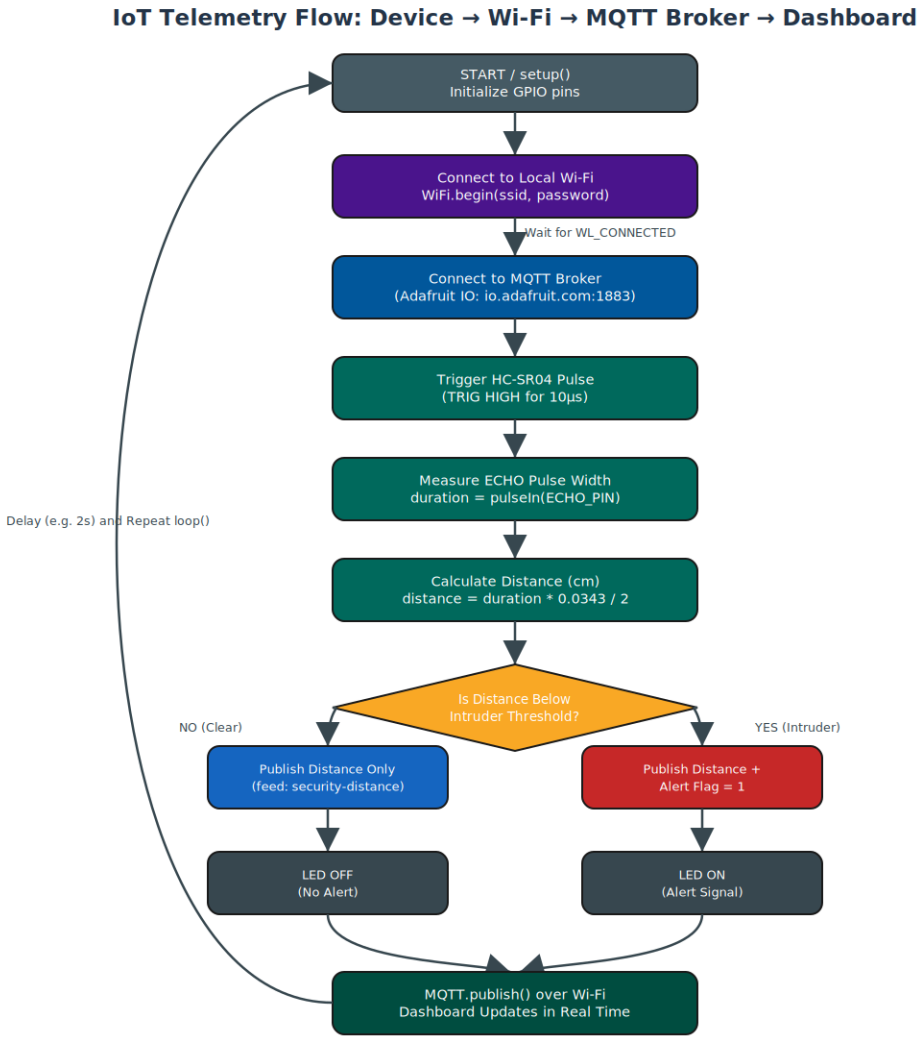

Figure 2: End-to-end telemetry flow implemented inside loop().

7. ESP32 Sketch (Wi-Fi + MQTT Telemetry)

The sketch below configures the Wi-Fi stack, connects to the Adafruit IO MQTT broker using the Adafruit MQTT library, reads the HC-SR04 sensor, and publishes the distance reading (plus an intruder alert flag) to two Adafruit IO feeds every few seconds.

```
// ================================================================
//  Project 3: Cloud-Connected Security Node (IoT Telemetry)
//  Board: ESP32 Dev Board
//  Libraries: WiFi.h, Adafruit_MQTT.h, Adafruit_MQTT_Client.h
// ================================================================

#include <WiFi.h>
#include "Adafruit_MQTT.h"
#include "Adafruit_MQTT_Client.h"

// --- Wi-Fi credentials ---
const char* WLAN_SSID = "YOUR_WIFI_SSID";
const char* WLAN_PASS = "YOUR_WIFI_PASSWORD";

// --- Adafruit IO (MQTT broker) credentials ---
#define AIO_SERVER      "io.adafruit.com"
#define AIO_SERVERPORT  1883
#define AIO_USERNAME    "YOUR_AIO_USERNAME"
#define AIO_KEY         "YOUR_AIO_KEY"

WiFiClient client;
Adafruit_MQTT_Client mqtt(&client, AIO_SERVER, AIO_SERVERPORT, AIO_USERNAME, AIO_KEY);
Adafruit_MQTT_Publish distanceFeed = Adafruit_MQTT_Publish(&mqtt, AIO_USERNAME "/feeds/security-
distance");
Adafruit_MQTT_Publish alertFeed    = Adafruit_MQTT_Publish(&mqtt, AIO_USERNAME "/feeds/security-
alert");

// --- Pin configuration ---
const int TRIG_PIN = 17;
const int ECHO_PIN = 16;
const int LED_PIN  = 18;

// --- Threshold logic ---
const float INTRUDER_THRESHOLD_CM = 30.0;

// --- Timing ---
const unsigned long PUBLISH_INTERVAL_MS = 2000;
unsigned long lastPublish = 0;

void connectWiFi() {
  WiFi.begin(WLAN_SSID, WLAN_PASS);
  Serial.print("Connecting to Wi-Fi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWi-Fi connected, IP address: " + WiFi.localIP().toString());
}

void connectMQTT() {
  int8_t ret;
```

```
  while ((ret = mqtt.connect()) != 0) {
    Serial.println(mqtt.connectErrorString(ret));
    mqtt.disconnect();
    delay(3000);
  }
  Serial.println("MQTT connected to Adafruit IO");
}

float readDistanceCM() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long duration = pulseIn(ECHO_PIN, HIGH, 30000); // 30ms timeout
  float distanceCM = duration * 0.0343 / 2.0;
  return distanceCM;
}

void setup() {
  Serial.begin(115200);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  connectWiFi();
  connectMQTT();
}

void loop() {
  // Keep MQTT connection alive
  if (!mqtt.connected()) {
    connectMQTT();
  }
  mqtt.processPackets(10);
  mqtt.ping();

  unsigned long now = millis();
  if (now - lastPublish >= PUBLISH_INTERVAL_MS) {
    lastPublish = now;

    float distance = readDistanceCM();
    bool intruderDetected = (distance < INTRUDER_THRESHOLD_CM);

    Serial.print("Distance: ");
    Serial.print(distance);
    Serial.println(" cm");

    // Publish distance telemetry
    distanceFeed.publish(distance);

    // Explicit threshold logic gate -> alert flag + local LED
    if (intruderDetected) {
      digitalWrite(LED_PIN, HIGH);
      alertFeed.publish(1);
      Serial.println("Status: INTRUDER DETECTED -> Alert Published");
    } else {
```

```
      digitalWrite(LED_PIN, LOW);
      alertFeed.publish(0);
      Serial.println("Status: CLEAR -> No Alert");
    }
  }
}
```

8. How the Telemetry System Works

8.1 Wi-Fi Stack Configuration

The ESP32's onboard radio is initialized with `WiFi.begin(ssid, password)`, which starts the association process with the local access point. The sketch blocks in a polling loop until `WiFi.status()` reports `WL_CONNECTED`, at which point the device has a local IP address and a route to the internet. This underlying TCP/IP stack is what allows the subsequent MQTT client to open a socket to a remote broker.

8.2 MQTT Telemetry Protocol

MQTT is a lightweight publish/subscribe messaging protocol designed for constrained devices and low-bandwidth, high-latency networks — making it ideal for IoT telemetry. The ESP32 acts as an MQTT client that opens a persistent connection to the Adafruit IO broker on port 1883. Rather than sending a full HTTP request for every reading (with its heavier headers and overhead), the device simply publishes a small payload to a named topic (a “feed”, e.g. `security-distance`). The broker instantly relays that payload to every subscriber, including the live dashboard.

8.3 Threshold Logic and Local Alerting

Alongside publishing raw telemetry, the sketch applies the same kind of explicit threshold gate used in actuator-based projects: if the measured distance falls below `INTRUDER_THRESHOLD_CM`, the device treats this as a potential intrusion. It immediately lights the local status LED for an at-a-glance physical indicator, and publishes an alert flag to a second feed so the cloud dashboard can distinguish routine distance readings from active security events.

8.4 Cloud Dashboard Integration

On the Adafruit IO side, each published feed is bound to a dashboard block (e.g. a live line chart for distance and a status indicator for the alert flag). Because MQTT pushes data to the broker in real time, the dashboard updates within a second or two of each publish call — giving a remote observer a live, continuously updating view of the protected zone without needing to poll the device directly.

9. Testing Notes (Simulation and Field Setup)

- In Tinkercad, the HC-SR04 sensor's simulated distance can be adjusted to trigger both the clear and intruder-detected branches of the threshold logic.
- Because Tinkercad's simulator does not provide real internet access, the Wi-Fi/MQTT portion of the sketch was validated on physical ESP32 hardware connected to a real access point and Adafruit IO account.
- The Serial Monitor was used to confirm successful Wi-Fi association, MQTT connection, and each published distance/alert value before checking the Adafruit IO dashboard.

- Recommended next step: add MQTT reconnect backoff and a watchdog timer so the node recovers automatically after a Wi-Fi drop.

End of Report — Week 3 Project, Decode Labs IoT Internship Program